Enzo Vidal Salvini
176269

1 - 4: Caderno

__/__/__ S T Q Q S S D

1. $A = \begin{bmatrix} 10 & -1 & 2 \\ -1 & 11 & -1 \\ 2 & -1 & 10 \end{bmatrix}$, $x = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$, $b = \begin{bmatrix} 6 \\ 22 \\ -10 \end{bmatrix}$

2. linha 1: $|10| > |-1| + |2|$ verdadeiro
linha 2: $|11| > |-1| + |-1|$ verdadeiro
linha 3: $|10| > |2| + |-1|$ verdadeiro

É importante para o método para garantir sua convergência para uma solução única, independentemente da escolha $x^{(0)}$

3.

| | |
|---|---|
| $x = \frac{1}{10} \cdot (6 + y - 2z)$ | $x^{(k)} = \frac{1}{10} (6 + y^{(k-1)} - 2z^{(k-1)}$ |
| $y = \frac{1}{11} \cdot (22 + x + z)$ | $y^{(k)} = \frac{1}{11} (22 + x^{(k)} + z^{(k-1)}$ |
| $z = \frac{1}{10} \cdot (-10 - 2x + y)$ | $z^{(k)} = \frac{1}{10} \cdot (-10 - 2x^{(k)} + y^{(k)}$ |

4. $x^{(0)} = (0,0,0)$

$\Rightarrow x^{(1)} = \frac{1}{10} \cdot (6 + y^{(0)} - 2z^{(0)}) = \frac{1}{10} (6) = \frac{6}{10} = 0,6$

$y^{(1)} = \frac{1}{11} (22 + x^{(1)} + z^{(0)}) = \frac{1}{11} \left(22 + \frac{6}{10} + 0\right) = \frac{22,6}{11} \approx 2,0545$

$z^{(1)} = \frac{1}{10} \cdot (-10 - 2x^{(1)} + y^{(1)}) \approx \frac{1}{10} \cdot (-10 - 2 \cdot 0,6 + 2,0545) \approx -0,9745$

$x^{(2)} = \frac{1}{10} (6 + y^{(1)} - 2z^{(1)}) \approx 0,9884$

$y^{(2)} = \frac{1}{11} (22 + x^{(2)} + z^{(1)}) \approx 2,0067$

$z^{(2)} = \frac{1}{10} \cdot (-10 - 2x^{(2)} + y^{(2)}) \approx -0,9970$

$x^{(3)} = \frac{1}{10} \cdot (6 + y^{(2)} - 2z^{(2)}) \approx 1,0001$

$y^{(3)} = \frac{1}{11} (22 + x^{(3)} + z^{(2)}) \approx 2,0003$

$z^{(3)} = \frac{1}{10} \cdot (-10 - 2x^{(3)} + y^{(3)}) \approx -0,99999$

5:

```
import numpy as np
import pandas as pd
def gauss_seidel(tol=1e-5, max_iter=100):
    # Condições iniciais
    x = 0.0
    y = 0.0
    z = 0.0

    historico = []

    # Adiciona iteração 0
    historico.append({
        'k': 0,
        'x^(k)': x,
        'y^(k)': y,
        'z^(k)': z,
        '||x^(k) - x^(k-1)||_inf': '-'
    })
    for k in range(1, max_iter + 1):
        x_old, y_old, z_old = x, y, z

        # Fórmulas de atualização
        x = (6.0 + y_old - 2.0 * z_old) / 10.0
        y = (22.0 + x + z_old) / 11.0
        z = (-10.0 - 2.0 * x + y) / 10.0

        # Erro (Norma Infinito)
        erro = max(abs(x - x_old), abs(y - y_old), abs(z - z_old))

        historico.append({
            'k': k,
            'x^(k)': x,
            'y^(k)': y,
            'z^(k)': z,
            '||x^(k) - x^(k-1)||_inf': erro
        })

        if erro < tol:
            break
```

```
    return pd.DataFrame(historico)
df_resultados = gauss_seidel()
print(df_resultados.to_string(index=False))
```

Iterações realizadas: 6
Solução aproximada: (x, y, z) = (1.000000, 2.000000, -1.000000)

6:

| k | x^(k) | y^(k) | z^(k) | \|\|x^(k) - x^(k-1)\|\|_inf |
|---|---|---|---|---|
| 0 | 0.000000 | 0.000000 | 0.000000 | - |
| 1 | 0.600000 | 2.054545 | -0.914545 | 2.054545 |
| 2 | 0.988364 | 2.006711 | -0.997002 | 0.388364 |
| 3 | 1.000071 | 2.000279 | -0.999986 | 0.011708 |
| 4 | 1.000025 | 2.000004 | -1.000005 | 0.000276 |
| 5 | 1.000001 | 2.000000 | -1.000000 | 0.000024 |
| 6 | 1.000000 | 2.000000 | -1.000000 | 0.000001 |

7:

```
import matplotlib.pyplot as plt

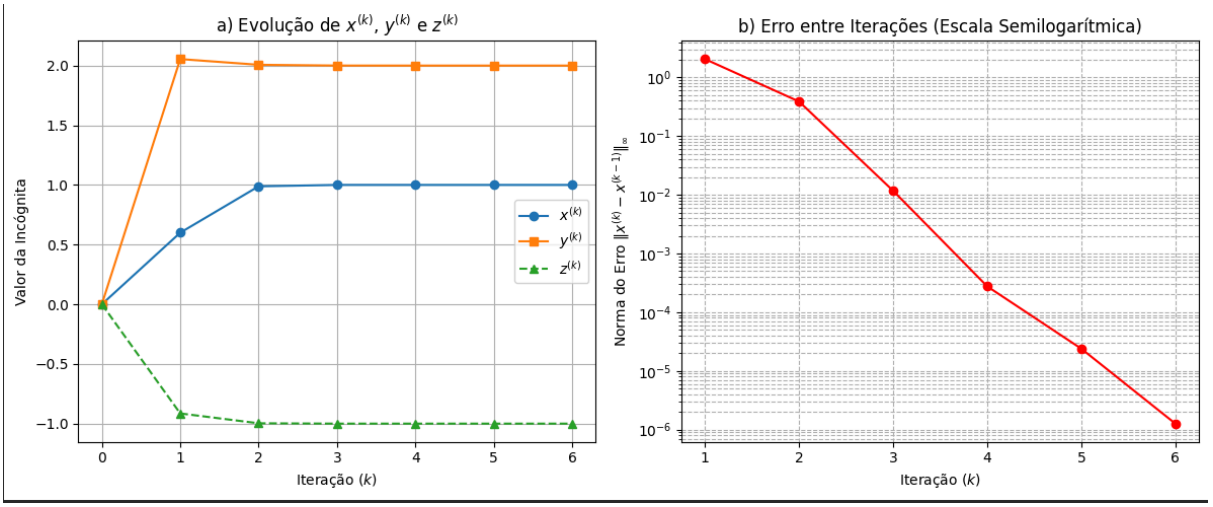
# Extração dos dados do DataFrame
k_vals = df_resultados['k']
x_vals = df_resultados['x^(k)']
y_vals = df_resultados['y^(k)']
z_vals = df_resultados['z^(k)']
erros = df_resultados['||x^(k) - x^(k-1)||_inf'].iloc[1:].astype(float)
k_erros = k_vals[1:]

plt.figure(figsize=(12, 5))

# Gráfico (a): Evolução das Variáveis
plt.subplot(1, 2, 1)
plt.plot(k_vals, x_vals, 'o-', label='x^(k)')
plt.plot(k_vals, y_vals, 's-', label='y^(k)')
plt.plot(k_vals, z_vals, '^--', label='z^(k)')
plt.xlabel('Iteração (k)')
plt.ylabel('Valor da Incógnita')
plt.title('a) Evolução de x^(k), y^(k) e z^(k)')
plt.grid(True)
plt.legend()

# Gráfico (b): Erro em Escala Logarítmica
plt.subplot(1, 2, 2)
plt.semilogy(k_erros, erros, 'ro-', linewidth=1.5)
```

```
plt.xlabel('Iteração (k)')
plt.ylabel(r'Norma do Erro $\|x^{(k)} - x^{(k-1)}\|_\infty$')
plt.title('b) Erro entre Iterações (Escala Semilogarítmica)')
plt.grid(True, which="both", ls="--")
plt.tight_layout()
plt.show()
```




Link do colab: Atividade19-10.ipynb